

2026-06-01 - KXTJ3-1057 CBOR Accelerometer on a Custom PCB

Goal

Deploy the KXTJ3-1057 CBOR-over-USB application from Day 16 onto a custom PCB board target instead of the nRF52840 DK. This teaches you how to move from a development kit to a production board definition.

What you will learn

- How to define a custom board that includes the KXTJ3-1057 as a named DTS node
- How to assign specific PCB pins in the Device Tree and pinctrl
- How to build and flash the full CBOR accelerometer app targeting `my_custom_board`
- How the board definition and application code stay cleanly separated

What changes compared to Day 16

Aspect	Day 16 (DK)	Day 17 (Custom PCB)
Build target	<code>nrf52840dk/nrf52840</code>	<code>my_custom_board</code>
Pin assignments	DK defaults	Your PCB schematic
KXTJ3 DTS node	<code>app.overlay</code>	<code>board.dts</code> file
USB D+/D- pins	DK defaults	PCB-specific pinctrl
<code>app.overlay</code>	adds KXTJ3 + USB node	only adds CDC-ACM shell node

Custom PCB assumptions

This example assumes an nRF52840-based custom PCB with:

Signal	nRF52840 GPIO	Reason
I2C SDA	P0.04	Chosen to avoid UART conflict
I2C SCL	P0.05	Adjacent to SDA
UART TX	P0.06	Debug UART to USB-serial chip
UART RX	P0.08	
LED	P1.10	Active-low status LED

Signal nRF52840 GPIO Reason

KXTJ3 INT P0.11 Optional data-ready interrupt

Update these to match your actual schematic before building.

Board definition highlights

my_custom_board.dts — KXTJ3 defined at board level

```
&i2c0 {
    status = "okay";
    kxtj3: kxtj3@e {
        compatible = "i2c-device";
        reg = <0x0e>;
        label = "KXTJ3";
    };
};
```

Because the KXTJ3 is soldered to the PCB, it belongs in the board .dts, not in the application overlay. The application overlay only adds the USB CDC-ACM shell node.

app.overlay — only USB CDC-ACM

```
/ {
    chosen {
        zephyr,console = &cdc_acm_uart0;
        zephyr,shell-uart = &cdc_acm_uart0;
    };
};

&zephyr_udc0 {
    cdc_acm_uart0: cdc_acm_uart0 {
        compatible = "zephyr,cdc-acm-uart";
        label = "CDC_ACM_0";
    };
};
```

Building for the custom board

```
# CMakeLists.txt sets BOARD_ROOT to pick up the local boards/ directory
west build -b my_custom_board
west flash
```

Python decoder (unchanged from Day 16)

```
pip install pyserial cbor2 matplotlib
python3 tools/decode_accel.py --port /dev/ttyACM0 --plot
```

Migration checklist

When moving any application from a DK to a custom board:

- [] Copy the closest upstream board DTS as a starting point
- [] Update all pin numbers to match your schematic
- [] Move any always-present peripherals (sensors, displays) from overlay to board DTS
- [] Keep application-specific overlays for things that could vary (USB, console routing)
- [] Verify `west build` succeeds with `-b my_custom_board`
- [] Check `build/zephyr/zephyr.dts` to confirm pin assignments are resolved correctly

Why this matters

Every shipped product has a custom board definition. Learning to separate board hardware description from application logic is the key skill that makes your firmware portable and maintainable across board revisions.

Example folder

See `src/2026-06-01_KXTJ3_CB0R_Custom_PCB` for the complete project.

Next topic

Future topics: BLE peripheral advertising, MCUboot OTA firmware updates, low-power wake-on-motion using KXTJ3 interrupt pin.